Coverings and Connections: A Mathematical Foundation for `covmap`

June 8, 2026

Abstract

`covmap` links two repositories A and B by partitioning each into content-addressed *segments* and relating those segments across repositories. This note gives precise, minimal definitions of repositories, segments, coverings, sharing of segments across coverings, and connections, then describes the command interface that realizes them. The implementation deliberately ships no segmentation *rule*: every covering is built up by hand, segment by segment, from a single whole-file bootstrap.

Contents

1 Segments and coverings **1**
- 1.1 Repositories . . . 1
- 1.2 Segments . . . 1
- 1.3 Coverings . . . 2
- 1.4 Multiple coverings and sharing . . . 2

2 Connections **3**

3 Complexity of a covering **3**
- 3.1 Cardinality and granularity . . . 3
- 3.2 Partition entropy . . . 3
- 3.3 Edit stability and drift . . . 4

4 An interface for `covmap` **4**
- 4.1 State on disk . . . 4
- 4.2 Addressing . . . 4
- 4.3 Bootstrap and refinement . . . 5
- 4.4 Inspection, connections, repair . . . 5
- 4.5 Plumbing . . . 6
- 4.6 Design discipline . . . 6

1 Segments and coverings

1.1 Repositories

Definition 1.1 (Repository)**.** *A* repository A *is a finite set of files, each a finite sequence of lines. We write* $|f|_A$ *for the number of lines of file* f *in* A*, and* $\mathrm{content}_A(f, i)$ *for the text of line* i *of file* f*.* `covmap` *considers only files whose bytes are valid UTF-8; other files are skipped on bootstrap and never appear in any covering.*

1.2 Segments

A segment is a contiguous range of lines inside a single file.

Definition 1.2 (Segment)**.** *Fix a set of* labels L*. A* segment *of A is a quadruple $s = (f, a, b, \lambda)$ where f is a file of A, $1 \le a \le b \le |f|_A$, and $\lambda \in L$. Its* lines *are*

$$\text{lines}(s) \;=\; \{(f, i) \mid a \le i \le b\},$$

its content *is the concatenation*

$$\text{content}_A(s) \;=\; \text{content}_A(f, a) \,\|\, \text{content}_A(f, a+1) \,\|\, \cdots \,\|\, \text{content}_A(f, b),$$

and its label *is* $\text{label}(s) = \lambda$*. We fix a hash function h and define the* content hash *of s as*

$$\text{hash}(s) \;=\; \text{trunc}_{16}\big(h\big(\lambda \,\|\, \texttt{NUL} \,\|\, \text{content}_A(s)\big)\big),$$

the first 16 hex characters (64 bits) of the SHA-256 of the labelled content. The truncated hash is the segment's identity.

Remark 1.3 (Why 64-bit identity)**.** *With n segments in play, the birthday-bound collision probability is approximately $n^2/2^{65}$. For repositories with up to a few tens of thousands of segments, this is negligible; the saving in display and storage compared to full SHA-256 is significant.*

The label is part of identity: two segments with identical content but different labels have different hashes and are therefore distinguishable as endpoints of a connection. `covmap` bootstraps every segment with the label `unknown`; typical re-labels include *ignored*, *reviewed*, and *mapped*, used to record per-segment status while preserving content-based sharing among segments that agree in both content and label.

Two segments $s_1 = (f, a_1, b_1, \lambda_1)$ and $s_2 = (f, a_2, b_2, \lambda_2)$ in the same file are *adjacent* if $b_1 + 1 = a_2$, and they *overlap* if their line ranges intersect (regardless of label).

1.3 Coverings

Definition 1.4 (Covering)**.** *Let $F \subseteq \text{files}(A)$. A* covering of A on F *is a finite set of segments $\mathcal{C} = \{s_1, \ldots, s_n\}$ such that, for every file $f \in F$, the segments of $\mathcal{C}$ that lie in f form a partition of f into contiguous blocks:*

(C1) ***Disjointness:*** *no two segments of $\mathcal{C}$ overlap;*

(C2) ***Completeness:*** *every line (f, i) with $f \in F$ and $1 \le i \le |f|_A$ lies in some segment of $\mathcal{C}$;*

(C3) ***Confinement:*** *every segment of $\mathcal{C}$ lies in some file of F.*

We call $F = F(\mathcal{C})$ the covered set *of $\mathcal{C}$.*

A covering need not partition the whole repository. It declares which files it accepts responsibility for. New files appearing in A are covered by re-running the bootstrap on $\mathcal{C}$ (§4.3), which is additive and never disturbs existing segments.

Equivalently, for each file $f \in F$ a covering supplies a sequence of cut points $1 = c_0 < c_1 < \cdots < c_{k_f} = |f|_A + 1$ together with a label $\lambda_j \in L$ for each block, and the segments in f are $(f, c_{j-1}, c_j - 1, \lambda_j)$.

Proposition 1.5 (Block lookup)**.** *For any covering $\mathcal{C}$ and any line (f, i) of A with $f \in F(\mathcal{C})$, exactly one segment of $\mathcal{C}$ contains (f, i).*

1.4 Multiple coverings and sharing

A repository may carry several coverings simultaneously. Different coverings often agree on some blocks. Because identity is content-addressed, agreement is automatic: two coverings $\mathcal{C}_1$ and $\mathcal{C}_2$ *share* a segment s exactly when $s \in \mathcal{C}_1$, $s \in \mathcal{C}_2$, and $\mathrm{hash}(s)$ is the same in both.

Definition 1.6 (Segment store)**.** *The* segment store *for A is a partial map*

$$\sigma_A : \{\textit{hashes}\} \rightharpoonup \{\textit{segments of } A\},$$

populated lazily as coverings are computed. A covering $\mathcal{C}$ is represented as a finite list of $(\mathrm{hash}(s), s)$ entries, and the segments themselves are fetched from the store by hash.

Definition 1.7 (DAG of coverings and segments)**.** *Given a family of coverings $\mathcal{C}_1, \ldots, \mathcal{C}_n$ of A, define a directed graph with one node per covering, one node per distinct segment hash that appears in some $\mathcal{C}_i$, and an edge $\mathcal{C}_i \to s$ whenever $s \in \mathcal{C}_i$.*

Proposition 1.8. *This graph is acyclic, and a segment that belongs to several coverings is represented exactly once.*

2 Connections

Definition 2.1 (Connection)**.** *Let $\mathcal{C}^A$ be a covering of A and $\mathcal{C}^B$ a covering of B. A* connection *from $\mathcal{C}^A$ to $\mathcal{C}^B$ is a relation*

$$\kappa \subseteq \mathcal{C}^A \times \mathcal{C}^B.$$

A pair $(s,t) \in \kappa$ records that segment s of A is linked to segment t of B. Connections are stored separately from coverings and refer to their endpoints by hash.

Definition 2.2 (Coverage)**.** *The* source coverage *of κ is*

$$\rho_{\mathrm{src}}(\kappa) = \frac{|\{s \in \mathcal{C}^A \mid \exists t.\ (s,t) \in \kappa\}|}{|\mathcal{C}^A|} \in [0,1].$$

The target coverage *is the symmetric quantity for $\mathcal{C}^B$. A connection is* source-total *if $\rho_{\mathrm{src}}(\kappa) = 1$.*

Definition 2.3 (Operations)**.** *Connections support*

- inverse: $\kappa^{-1} = \{(t,s) \mid (s,t) \in \kappa\}$;
- composition: *for $\kappa \subseteq \mathcal{C}^A \times \mathcal{C}^B$ and $\lambda \subseteq \mathcal{C}^B \times \mathcal{C}^C$,* $\lambda \circ \kappa = \{(s,u) \mid \exists t.\ (s,t) \in \kappa,\ (t,u) \in \lambda\}$;
- union *of two connections with the same endpoints.*

3 Complexity of a covering

A covering can be characterized by several quantitative measures.

Let $\ell(s) = b - a + 1$ be the *length* of a segment $s = (f, a, b)$, and let $N_{\mathcal{C}} = \sum_{s \in \mathcal{C}} \ell(s)$ be the total number of covered lines.

3.1 Cardinality and granularity

Definition 3.1 (Cardinality, granularity)**.** *The* cardinality *of $\mathcal{C}$ is $|\mathcal{C}|$, and its* granularity *is*

$$g(\mathcal{C}) = \frac{|\mathcal{C}|}{N_{\mathcal{C}}} \in (0,1].$$

A bootstrap covering (one segment per covered file) has g close to 0; the one-line-per-segment covering has $g = 1$.

3.2 Partition entropy

Definition 3.2 (Partition entropy)**.** *Let* $p_{\mathcal{C}}(s) = \ell(s)/N_{\mathcal{C}}$*. The* entropy *of* $\mathcal{C}$ *is*

$$H(\mathcal{C}) \;=\; -\sum_{s\in\mathcal{C}} p_{\mathcal{C}}(s)\log_2 p_{\mathcal{C}}(s).$$

Entropy ranges from 0 (a single block covering one one-line file) to $\log_2 N_{\mathcal{C}}$ (one block per line). Coverings with equal-length blocks maximize entropy at fixed cardinality.

3.3 Edit stability and drift

A covering exists to be reused after the repository is edited. We measure how stable a covering is under edits.

Definition 3.3 (File-level drift)**.** *Let* $\mathcal{C}$ *be a covering of* A *and let* A' *be a repository obtained from* A *by an edit. A file* $f \in F(\mathcal{C})$ *is* drifted *under this edit when at least one segment* $s = (f, a, b, \lambda) \in \mathcal{C}$ *satisfies* $\text{content}_{A'}(s) \neq \text{content}_A(s)$*.*

Definition 3.4 (Boundary drift)**.** *A drifted file* f *has* boundary drift *if some segment of* $\mathcal{C}$ *in* f *has* $b > |f|_{A'}$*, i.e., the recorded line range no longer fits the file.*

The two flavours of drift correspond to two repair operations (§4): `recut` for file-level drift with intact boundaries, `merge` + `split` for boundary drift.

Definition 3.5 (Broken pair)**.** *A pair* $(s, t) \in \kappa$ *is* broken *relative to current* $\mathcal{C}^A$ *and* $\mathcal{C}^B$ *when* $\text{hash}(s) \notin \text{hash}(\mathcal{C}^A)$ *or* $\text{hash}(t) \notin \text{hash}(\mathcal{C}^B)$*.*

A pair becomes broken precisely when its endpoint is rewritten (`recut`, `label`, `split`, `merge`) or removed.

4 An interface for `covmap`

The model above suggests a small command surface, organized `git`-style around an object store, lightweight named refs, and human-readable text refs.

4.1 State on disk

A repository under `covmap` acquires a `.covmap/` directory:

```
.covmap/
  HEAD                 default covering name
  ignore               optional gitignore-lite patterns
  objects/<hh>/<rest>  segments, addressed by hash
  coverings/<name>     one file per covering (covered files + segments)
  connections/<name>   one file per connection (metadata + pairs)
```

4.2 Addressing

Every object has a stable address. Three address forms appear throughout the interface and can be mixed freely.

`<hash>`	a segment, by full or short hex prefix
C`:`h	a segment h in covering C
K	a connection by name

A bare h is short for `HEAD:`h. Where a covering is expected, the literal `HEAD` resolves to the default covering recorded in `.covmap/HEAD`. Hash prefixes follow `git`-style disambiguation: any unique prefix works, ambiguity is an error.

4.3 Bootstrap and refinement

There is no segmentation rule. The interface gives the user one bootstrap operation and two refinement operations.

- `covmap init`
  Create `.covmap/` in the current directory.
- `covmap cover` $[C]$
  For every readable UTF-8 file under the repo not already covered by C, add a single segment $(f, 1, |f|_A, \texttt{unknown})$ to C. Idempotent: re-running `cover` adds segments only for new files. Existing segments are never touched, so any user-driven refinement is preserved.
- `covmap split` $C{:}h\ c$ `[`λ_t`]` `[`λ_b`]`
  Replace the segment with hash h by two adjacent segments $(f, a, c-1, \lambda_t)$ and (f, c, b, λ_b), where $a \le c \le b$ and $a < c$. Both new labels default to `unknown`.
- `covmap merge` $C{:}h_1\ C{:}h_2$ `[`λ`]`
  Replace two adjacent segments by their union, labelled λ (default `unknown`).
- `covmap recut` $C{:}h$
  Refresh the segment's hash from current disk bytes. Keeps (f, a, b, λ); rewrites the content from the file as it is now. Errors if the segment no longer fits the file (boundary drift); use `merge`+`split` in that case.
- `covmap label` $C{:}h\ \lambda$
  Replace the label of one segment. Because labels are part of identity, this rewrites the entry in C; old hashes remain in the object store so any connection that referenced them keeps resolving.

These five operations are the only way to change a covering after `cover`. Together they are complete: any covering reachable from the bootstrap by repeated splits / merges / recuts / relabels is the most general covering one can ask for.

4.4 Inspection, connections, repair

- `covmap status` $[C]$
  One-line summary per covering: cardinality, line count, entropy. A drift line reports the number of files whose bytes have changed since their segments were recorded. Broken pair counts are reported per connection.
- `covmap show <addr>`
  Print the segment, covering, or connection at `<addr>`.
- `covmap ls [-l]` C
  List segments of C in file-and-line order.
- `covmap grep [-l] [-c` C`] <pattern>`
  Print segments whose label or content matches `<pattern>`.
- `covmap diff` $C_1\ C_2$
  Segments only in C_1, only in C_2, and shared.
- `covmap connect` $C_A\ \langle\texttt{repo}_B\rangle{:}C_B$ `[`K`]`
  Create an empty connection between two coverings.
- `covmap link` $K\ h_A\ h_B$ `[`ν`]`
  `covmap unlink` $K\ h_A\ h_B$
  Add or remove a single pair, optionally annotated with note ν.

- `covmap repair` K
  Drop every pair of K whose endpoint is no longer in its covering.
- `covmap fsck`
  Verify that every covering is a partition of its covered set and that every connection's endpoints resolve.

4.5 Plumbing

Lower-level commands meant for scripts.

- `covmap hash-segment <file> <a> <b> [<label>] [-w]`
- `covmap cat-segment <hash>`
- `covmap ls-segments` C
- `covmap ls-pairs` K
- `covmap rev-parse <addr>`
- `covmap fsck -broken-pairs`

4.6 Design discipline

- Coverings and connections are independent artifacts; a connection refers to its coverings only by hash.
- Every state-changing operation is explicit. `cover` is additive only. Refining a covering requires `split`, `merge`, `recut`, or `label` — `covmap` never silently re-segments existing files.
- The plumbing/porcelain split is hard: every porcelain command is implementable as a shell pipeline of plumbing commands.
- Hashes are the lingua franca. Every command accepts them on input and prints them on output, so commands compose without glue code.